

NEXTURN

Data Engineering Training

Scala and Apache Spark End-to-End Multi-Domain Booking Payment Platform

Technical Documentation

Name: Harshita Pendli

Role: Data Engineering Trainee

Company: NexTurn India Private Limited

Project: End-to-End Multi-Domain Booking Payment Platform

Technologies: Scala, Apache Spark, Hadoop, YARN

September 2026

Contents

1	Introduction	3
2	Project Objectives	3
3	Project Setup and Directory Structure	4
3.1	Project Directory Creation	4
4	Dataset Overview	5
5	Bookings Dataset	5
5.1	Bookings Schema Verification	6
5.2	Bookings Data	6
6	Bus Dataset	7
7	Flight Dataset	8
8	Hotel Dataset	9
9	Cancellations Dataset	10
10	Payments Dataset	10
11	Booking Data Verification	11
12	Dataset Record Counts	12
13	Dataset Relationship Overview	13
14	Data Validation	13
15	Technologies Used	14
16	Summary of Completed Data Preparation	14
17	Results and Analysis	15
17.1	Booking Summary by Type	15
17.2	Customer Booking Analysis	16
17.3	Customer-Wise Spending Analysis	16
17.4	Payment Analysis	17
17.5	Payment Summary	18
17.6	Cancellation Analysis	18

17.7 Cancellation Summary and Hotel Booking Analysis	18
17.8 Hotel Summary and Flight Booking Analysis	19
17.9 Flight Summary and Bus Booking Analysis	19
17.10Bus Summary	20
17.11Physical Execution Plan	21
18 Advanced Spark Concepts and Streaming	21
18.1 Pair RDD Customer Revenue	22
18.2 Narrow Transformation, Partitioning and Wide Transformation	22
18.3 Cache, Persist, Broadcast Variable and Accumulator	23
18.4 Spark SQL Aggregation	23
18.5 Spark SQL Top 5 Bookings	24
18.6 Customer Spending Rank	25
18.7 Customer Rank by City	25
18.8 User Defined Function for Booking Classification	26
18.9 RDD Lineage and DAG Execution	27
18.10Structured Streaming Initialization	27
19 Final Case Study Verification	27
19.1 Scala Fundamentals and Dataset Verification	27
19.2 Spark Catalog and Spark SQL	29
19.3 Spark Monitoring and YARN Deployment	30
19.4 Streaming Application Execution	31
20 Final Verification and Monitoring	34
20.1 Final Verification Output	34
20.2 Spark Storage Monitoring	35
20.3 Spark Executors Monitoring	36
20.4 YARN Deployment Verification	36
20.5 Spark Monitoring Port and YARN	37
21 Conclusion	38

1. Introduction

This case study was developed as part of the Data Engineering training at Nexturn. The objective of the project is to understand and implement data processing using Scala and Apache Spark through a practical booking and payment platform scenario.

The project uses multiple related datasets representing customers, bookings, payments, cancellations, hotels, flights, and buses. These datasets are processed using Spark to understand schema definition, data loading, transformations, actions, joins, aggregations, caching, Spark monitoring, streaming concepts, and YARN-based deployment.

The implementation was performed in a Linux environment using Scala and Apache Spark. Hadoop and YARN were also configured to demonstrate cluster-oriented execution.

2. Project Objectives

The major objectives and deliverables of this case study are:

- To understand and demonstrate Scala programming fundamentals, including collections, for-yield expressions, lazy evaluation, and traits.
- To create and work with structured booking datasets using Apache Spark.
- To define, inspect, and validate Dataset schemas and records.
- To process booking and payment-related data containing booking ID, customer ID, booking type, service ID, booking date, amount, and status.
- To perform Spark transformations and actions for data processing.
- To implement filtering and conditional processing to identify confirmed bookings.
- To perform joins and pair RDD operations for customer-wise revenue analysis.
- To perform aggregations for booking-type revenue, customer revenue, and booking counts.
- To create and inspect temporary tables using the Spark Catalog.
- To perform analytical queries using Spark SQL.
- To implement DStream-based micro-batch streaming for continuous processing of booking records.

- To implement stateless transformations for processing each streaming batch independently.
- To implement stateful processing to maintain customer revenue across streaming batches.
- To implement time-based window operations for revenue and booking-count analysis.
- To understand Spark execution through jobs, stages, tasks, RDDs, storage, and executors.
- To monitor the Spark application using the Spark Web UI and verify application execution through port 4040.
- To configure and verify Hadoop YARN for cluster resource management.
- To verify the YARN environment using the YARN node list.
- To prepare the Spark application for YARN-based cluster execution.
- To perform final compilation, execution, and verification of all implemented components.

3. Project Setup and Directory Structure

The project directory was created with separate locations for input data, reference materials, output data, and Scala source code.

The following directories were created:

- `data/input` – used for input datasets.
- `data/reference` – used for reference materials.
- `data/output` – used for generated output and analysis results.
- `src/main/scala` – used for Scala and Spark source code.

3.1 Project Directory Creation

The following screenshot shows the initial creation of the project directories.

```
harshita@harshita:~$ mkdir -p ~/booking-payment-platform/data/input
harshita@harshita:~$ mkdir -p ~/booking-payment-platform/data/reference
harshita@harshita:~$ mkdir -p ~/booking-payment-platform/data/output
harshita@harshita:~$ mkdir -p ~/booking-payment-platform/src/main/scala
harshita@harshita:~$ cd ~/booking-payment-platform
```

Figure 1: Creation of project directories

The directory structure separates source code, input data, reference material, and output results. This provides an organized structure for developing and executing the case study.

4. Dataset Overview

The booking and payment platform contains multiple datasets representing different business entities and transactions.

The datasets used in the case study include:

- Customers
- Bookings
- Payments
- Cancellations
- Hotels
- Flights
- Buses

These datasets represent different parts of the booking ecosystem. The **bookings** dataset acts as an important transactional dataset because bookings are associated with customers and services such as hotels, flights, and buses.

5. Bookings Dataset

The bookings dataset contains the transaction-level booking information.

The schema contains the following attributes:

- **booking_id** – unique identifier of the booking.
- **customer_id** – identifies the customer who made the booking.
- **booking_type** – identifies whether the booking is for a hotel, flight, or bus.

- `service_id` – identifies the corresponding service.
- `booking_date` – date on which the booking was made.
- `amount` – monetary amount associated with the booking.
- `status` – current status of the booking.

5.1 Bookings Schema Verification

The following screenshot shows the schema generated and displayed using Spark.

```
=== BOOKINGS SCHEMA ===
root
 |-- booking_id: string (nullable = true)
 |-- customer_id: string (nullable = true)
 |-- booking_type: string (nullable = true)
 |-- service_id: string (nullable = true)
 |-- booking_date: date (nullable = true)
 |-- amount: integer (nullable = true)
 |-- status: string (nullable = true)

=== TOTAL BOOKINGS ===
15
```

Figure 2: Bookings DataFrame schema and record count

The output confirms that the bookings DataFrame contains seven attributes. The displayed total number of booking records is 15.

5.2 Bookings Data

The complete booking data was loaded and displayed using Spark.

```

=== BOOKINGS DATA ===
+-----+-----+-----+-----+-----+-----+-----+
|booking_id|customer_id|booking_type|service_id|booking_date|amount|status|
+-----+-----+-----+-----+-----+-----+-----+
|BK001|C001|Hotel|H001|2026-09-01|10000|CONFIRMED|
|BK002|C002|Flight|F001|2026-09-01|6500|CONFIRMED|
|BK003|C003|Bus|B002|2026-09-01|1200|CONFIRMED|
|BK004|C004|Hotel|H002|2026-09-02|15000|CONFIRMED|
|BK005|C005|Flight|F002|2026-09-02|7200|CANCELLED|
|BK006|C006|Bus|B001|2026-09-02|1000|CONFIRMED|
|BK007|C007|Hotel|H003|2026-09-03|9000|CONFIRMED|
|BK008|C008|Flight|F003|2026-09-03|5800|CONFIRMED|
|BK009|C009|Bus|B003|2026-09-03|1500|CANCELLED|
|BK010|C010|Hotel|H004|2026-09-04|13000|CONFIRMED|
|BK011|C001|Flight|F004|2026-09-04|8000|CONFIRMED|
|BK012|C002|Bus|B004|2026-09-04|900|CONFIRMED|
|BK013|C003|Hotel|H005|2026-09-05|7000|CONFIRMED|
|BK014|C004|Flight|F005|2026-09-05|6200|CONFIRMED|
|BK015|C005|Bus|B005|2026-09-05|1100|CANCELLED|
+-----+-----+-----+-----+-----+-----+-----+

```

Figure 3: Bookings dataset displayed using Spark

The dataset contains booking IDs from BK001 to BK015. Each record contains the customer, booking type, service ID, booking date, amount, and booking status.

The data contains different booking categories such as:

- Hotel
- Flight
- Bus

The booking status includes values such as CONFIRMED and CANCELLED.

6. Bus Dataset

The bus dataset represents bus services available in the booking platform.

It contains:

- bus_id
- operator
- source
- destination
- seat_capacity


```

=== BUSES ===
+-----+-----+-----+-----+-----+
|bus_id|operator|source  |destination|seat_capacity|
+-----+-----+-----+-----+-----+
|B001  |TSRTC   |Hyderabad|Bangalore  |40           |
|B002  |KSRTC   |Bangalore|Chennai    |45           |
|B003  |APSRTC  |Hyderabad|Chennai    |50           |
|B004  |MSRTC   |Mumbai   |Pune       |40           |
|B005  |KSRTC   |Chennai  |Bangalore  |45           |
+-----+-----+-----+-----+-----+

```

Figure 4: Bus dataset

The displayed data contains five bus records. The records represent different operators and routes such as Hyderabad–Bangalore, Bangalore–Chennai, Hyderabad–Chennai, Mumbai–Pune, and Chennai–Bangalore.

The `seat_capacity` attribute represents the available capacity of each bus service.

7. Flight Dataset

The flight dataset represents flight services available through the platform.

The dataset contains:

- `flight_id`
- `airline`
- `source`
- `destination`
- `seat_capacity`

```

=== FLIGHTS ===
+-----+-----+-----+-----+-----+
|flight_id|airline |source  |destination|seat_capacity|
+-----+-----+-----+-----+-----+
|F001     |IndiGo  |Hyderabad|Delhi      |180          |
|F002     |AirIndia|Bangalore|Mumbai     |160          |
|F003     |IndiGo  |Chennai  |Hyderabad  |180          |
|F004     |Vistara |Delhi    |Goa        |150          |
|F005     |AirIndia|Mumbai   |Bangalore  |160          |
+-----+-----+-----+-----+-----+

```

Figure 5: Flight dataset

The screenshot shows five flight records belonging to airlines including IndiGo, Air India, and Vistara.

The source and destination attributes represent the route, while `seat_capacity` represents the number of seats available for the corresponding flight.

8. Hotel Dataset

The hotel dataset represents hotel services available in the booking platform.

The attributes include:

- `hotel_id`
- `hotel_name`
- `city`
- `room_type`
- `price_per_night`

```

=== HOTELS ===
+-----+-----+-----+-----+-----+
|hotel_id|hotel_name  |city    |room_type|price_per_night|
+-----+-----+-----+-----+-----+
|H001    |Taj Residency|Hyderabad|Deluxe   |5000           |
|H002    |Grand Palace |Bangalore|Suite    |7500           |
|H003    |Marina Hotel  |Chennai  |Deluxe   |4500           |
|H004    |Sea View Resort|Goa      |Premium  |6500           |
|H005    |Royal Inn     |Mumbai   |Standard |3500           |
+-----+-----+-----+-----+-----+

```

Figure 6: Hotel dataset

The displayed dataset contains five hotel records. Each record identifies the hotel, its city, room type, and price per night.

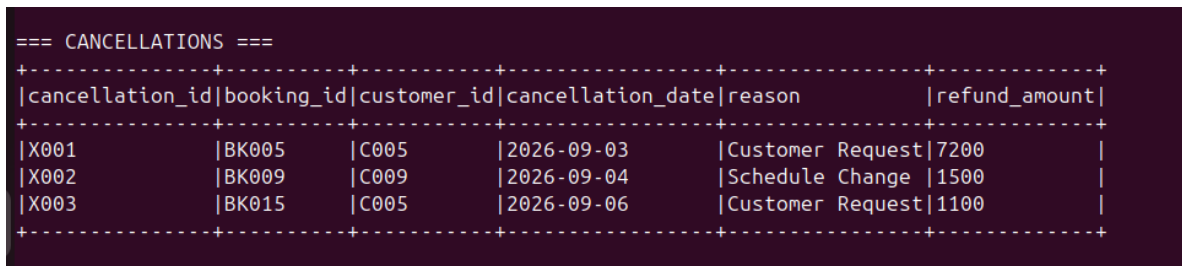
The dataset demonstrates how service-specific reference information can be maintained separately from the transactional bookings data.

9. Cancellations Dataset

The cancellations dataset contains information about cancelled bookings.

The attributes shown are:

- `cancellation_id`
- `booking_id`
- `customer_id`
- `cancellation_date`
- `reason`
- `refund_amount`



```
=== CANCELLATIONS ===
+-----+-----+-----+-----+-----+-----+
|cancellation_id|booking_id|customer_id|cancellation_date|reason          |refund_amount|
+-----+-----+-----+-----+-----+-----+
|X001           |BK005    |C005      |2026-09-03      |Customer Request|7200         |
|X002           |BK009    |C009      |2026-09-04      |Schedule Change |1500         |
|X003           |BK015    |C005      |2026-09-06      |Customer Request|1100         |
+-----+-----+-----+-----+-----+-----+
```

Figure 7: Cancellations dataset

The screenshot shows three cancellation records. The cancellation data is linked to bookings and customers through their respective identifiers.

The `reason` attribute captures why the cancellation occurred, while `refund_amount` represents the corresponding refund amount.

10. Payments Dataset

The payments dataset represents payment transactions associated with bookings.

The attributes include:

- `payment_id`

- booking_id
- payment_date
- payment_method
- payment_amount
- payment_status

```

=== PAYMENTS ===
+-----+-----+-----+-----+-----+-----+
|payment_id|booking_id|payment_date|payment_method|payment_amount|payment_status|
+-----+-----+-----+-----+-----+-----+
|P001      |BK001     |2026-09-01  |UPI           |10000         |SUCCESS       |
|P002      |BK002     |2026-09-01  |CARD          |6500          |SUCCESS       |
|P003      |BK003     |2026-09-01  |UPI           |1200          |SUCCESS       |
|P004      |BK004     |2026-09-02  |CARD          |15000         |SUCCESS       |
|P005      |BK005     |2026-09-02  |UPI           |7200          |REFUNDED      |
+-----+-----+-----+-----+-----+-----+
only showing top 5 rows

```

Figure 8: Payments dataset

The displayed records demonstrate different payment methods such as UPI and CARD. Payment statuses include SUCCESS and REFUNDED.

The booking_id provides the relationship between the payment transaction and its corresponding booking.

11. Booking Data Verification

A separate output was also generated to verify that the booking data could be correctly loaded and displayed through Spark.

```

=== BOOKINGS ===
+-----+-----+-----+-----+-----+-----+
|booking_id|customer_id|booking_type|service_id|booking_date|amount|status  |
+-----+-----+-----+-----+-----+-----+
|BK001     |C001       |Hotel      |H001      |2026-09-01  |10000  |CONFIRMED|
|BK002     |C002       |Flight     |F001      |2026-09-01  |6500   |CONFIRMED|
|BK003     |C003       |Bus        |B002      |2026-09-01  |1200   |CONFIRMED|
|BK004     |C004       |Hotel      |H002      |2026-09-02  |15000  |CONFIRMED|
|BK005     |C005       |Flight     |F002      |2026-09-02  |7200   |CANCELLED|
+-----+-----+-----+-----+-----+-----+
only showing top 5 rows

```

Figure 9: Sample booking records displayed using Spark

The output displays the first five booking records. It confirms that the booking DataFrame contains the expected columns and that the records have been successfully loaded into Spark.

The displayed records include different service types such as Hotel, Flight, and Bus, together with their corresponding service identifiers and booking amounts.

12. Dataset Record Counts

The record-count verification was performed for all major datasets.

```
=== DATASET RECORD COUNTS ===
Customers      : 10
Bookings       : 15
Payments       : 15
Cancellations  : 3
Hotels          : 5
Flights         : 5
Buses          : 5

=== CUSTOMERS ===
+-----+-----+---+-----+
|customer_id|name  |age|city      |
+-----+-----+---+-----+
|C001       |Harsha|24 |Hyderabad|
|C002       |Rahul |29 |Bangalore|
|C003       |Ananya|26 |Chennai  |
|C004       |Vikram|35 |Mumbai   |
|C005       |Meera |31 |Delhi    |
+-----+-----+---+-----+
only showing top 5 rows
```

Figure 10: Dataset record counts and customer data

The displayed record counts are:

Dataset	Record Count
Customers	10
Bookings	15
Payments	15
Cancellations	3
Hotels	5
Flights	5
Buses	5

Table 1: Dataset record counts

The customer dataset contains customer identifiers, names, ages, and cities. The screenshot displays the first five customer records.

This verification step is important because it confirms that the expected number of records has been loaded into the Spark application before further processing and analysis.

13. Dataset Relationship Overview

The datasets represent different entities and transactions in the booking platform.

The main transactional relationship can be understood through the `booking_id` and `customer_id` fields.

- Customers are identified using `customer_id`.
- Bookings contain `customer_id` to associate a booking with a customer.
- Payments contain `booking_id` to associate a payment with a booking.
- Cancellations contain `booking_id` and `customer_id` to associate cancellations with bookings and customers.
- Bookings contain `service_id`, which identifies the corresponding hotel, flight, or bus service depending on the booking type.

This structure provides the foundation for performing joins and analytical queries using Apache Spark.

14. Data Validation

Before performing advanced transformations, the datasets were verified by checking their schemas, record counts, and sample records.

The validation process included:

1. Loading the datasets into Spark.
2. Printing the schema of the bookings dataset.
3. Checking the total number of records.
4. Displaying sample booking records.
5. Displaying reference datasets such as hotels, flights, and buses.
6. Checking payment records.
7. Checking cancellation records.
8. Checking customer records.

The screenshots in this section provide evidence that the datasets were successfully loaded and that the expected structures and record counts were obtained.

15. Technologies Used

The case study uses the following technologies:

- **Scala** – programming language used for implementation.
- **Apache Spark** – distributed data processing framework.
- **Spark SQL/DataFrames** – structured data processing.
- **Spark Streaming** – processing of continuously arriving data.
- **Hadoop** – distributed storage and cluster infrastructure.
- **YARN** – cluster resource management and application deployment.
- **Linux/Ubuntu** – development and execution environment.

16. Summary of Completed Data Preparation

At this stage of the implementation, the project environment and basic data layer were established.

The following activities were completed:

- Created the project directory structure.
- Created separate folders for input, reference, and output data.

- Created the Scala source directory.
- Loaded the booking-platform datasets into Spark.
- Verified the bookings schema.
- Verified booking record count.
- Displayed booking records.
- Loaded and verified bus reference data.
- Loaded and verified flight reference data.
- Loaded and verified hotel reference data.
- Loaded and verified cancellation data.
- Loaded and verified payment data.
- Loaded and verified customer data.
- Verified dataset record counts.
- Confirmed the relationships required for subsequent Spark processing.

These steps establish the data foundation for the remaining Scala and Spark processing activities in the case study.

17. Results and Analysis

The implemented Apache Spark and Scala pipeline was executed successfully on the Booking-Payment Platform dataset. The results below show the outputs generated from the Spark DataFrame operations, joins, aggregations, grouping, sorting, filtering, and analytical queries performed on the booking, customer, payment, cancellation, hotel, flight, and bus datasets.

17.1 Booking Summary by Type

The booking summary groups the bookings according to booking type and calculates the total number of bookings and total revenue generated by each category. The output shows 5 bookings for each of Bus, Flight, and Hotel, with Hotel generating the highest revenue.


```

=== BOOKING SUMMARY BY TYPE ===
+-----+-----+-----+
|booking_type|total_bookings|total_revenue|
+-----+-----+-----+
|Bus          |5             |5700          |
|Flight       |5             |33700         |
|Hotel        |5             |54000         |
+-----+-----+-----+

[success] Total time: 47 s, completed Sep 3, 2026 10:47:57 AM

```

Figure 11: Spark output for booking summary by booking type

17.2 Customer Booking Analysis

The customer booking analysis combines customer and booking information using the customer ID. It displays the booking ID, customer details, booking type, amount, and booking status. This demonstrates the use of an inner join between the customer and booking datasets.

```

=== CUSTOMER BOOKING ANALYSIS ===
+-----+-----+-----+-----+-----+-----+-----+
|booking_id|customer_id|name  |city    |booking_type|amount|status |
+-----+-----+-----+-----+-----+-----+-----+
|BK001     |C001       |Harsha|Hyderabad|Hotel        |10000 |CONFIRMED|
|BK011     |C001       |Harsha|Hyderabad|Flight       |8000  |CONFIRMED|
|BK002     |C002       |Rahul |Bangalore|Flight       |6500  |CONFIRMED|
|BK012     |C002       |Rahul |Bangalore|Bus          |900   |CONFIRMED|
|BK003     |C003       |Ananya|Chennai  |Bus          |1200  |CONFIRMED|
|BK013     |C003       |Ananya|Chennai  |Hotel        |7000  |CONFIRMED|
|BK004     |C004       |Vikram|Mumbai   |Hotel        |15000 |CONFIRMED|
|BK014     |C004       |Vikram|Mumbai   |Flight       |6200  |CONFIRMED|
|BK005     |C005       |Meera |Delhi    |Flight       |7200  |CANCELLED|
|BK015     |C005       |Meera |Delhi    |Bus          |1100  |CANCELLED|
|BK006     |C006       |Arjun |Pune     |Bus          |1000  |CONFIRMED|
|BK007     |C007       |Sneha |Kochi    |Hotel        |9000  |CONFIRMED|
|BK008     |C008       |Kiran |Hyderabad|Flight       |5800  |CONFIRMED|
|BK009     |C009       |Priya |Bangalore|Bus          |1500  |CANCELLED|
|BK010     |C010       |Rohit |Chennai  |Hotel        |13000 |CONFIRMED|
+-----+-----+-----+-----+-----+-----+-----+

[success] Total time: 25 s, completed Sep 3, 2026 10:52:39 AM

```

Figure 12: Customer booking analysis using Spark join operation

17.3 Customer-Wise Spending Analysis

Customer-wise spending is calculated by grouping booking records by customer and aggregating the number of bookings and total spending. The results are sorted in descending

order of total spending. Vikram has the highest total spending of 21,200, followed by Harsha with 18,000.

```

=== CUSTOMER-WISE SPENDING ===
+-----+-----+-----+-----+
|customer_id|name  |total_bookings|total_spending|
+-----+-----+-----+-----+
|C004       |Vikram|2             |21200         |
|C001       |Harsha|2             |18000         |
|C010       |Rohit |1             |13000         |
|C007       |Sneha |1             |9000          |
|C005       |Meera |2             |8300          |
|C003       |Ananya|2             |8200          |
|C002       |Rahul |2             |7400          |
|C008       |Kiran |1             |5800          |
|C009       |Priya |1             |1500          |
|C006       |Arjun |1             |1000          |
+-----+-----+-----+-----+
[success] Total time: 22 s, completed Sep 3, 2026 10:55:17 AM

```

Figure 13: Customer-wise booking count and total spending

17.4 Payment Analysis

The payment analysis combines booking and payment information using the booking ID. It displays booking amount, booking status, payment method, payment amount, and payment status. The output demonstrates how payment information can be related to individual bookings.

```

=== PAYMENT ANALYSIS ===
+-----+-----+-----+-----+-----+-----+-----+-----+
|booking_id|customer_id|booking_type|booking_amount|booking_status|payment_method|payment_amount|payment_status|
+-----+-----+-----+-----+-----+-----+-----+-----+
|BK001     |C001       |Hotel       |10000         |CONFIRMED     |UPI           |10000         |SUCCESS       |
|BK002     |C002       |Flight      |6500          |CONFIRMED     |CARD          |6500          |SUCCESS       |
|BK003     |C003       |Bus         |1200          |CONFIRMED     |UPI           |1200          |SUCCESS       |
|BK004     |C004       |Hotel       |15000         |CONFIRMED     |CARD          |15000         |SUCCESS       |
|BK005     |C005       |Flight      |7200          |CANCELLED     |UPI           |7200          |REFUNDED     |
|BK006     |C006       |Bus         |1000          |CONFIRMED     |CARD          |1000          |SUCCESS       |
|BK007     |C007       |Hotel       |9000          |CONFIRMED     |UPI           |9000          |SUCCESS       |
|BK008     |C008       |Flight      |5800          |CONFIRMED     |CARD          |5800          |SUCCESS       |
|BK009     |C009       |Bus         |1500          |CANCELLED     |UPI           |1500          |REFUNDED     |
|BK010     |C010       |Hotel       |13000         |CONFIRMED     |CARD          |13000         |SUCCESS       |
|BK011     |C001       |Flight      |8000          |CONFIRMED     |UPI           |8000          |SUCCESS       |
|BK012     |C002       |Bus         |900           |CONFIRMED     |CARD          |900           |SUCCESS       |
|BK013     |C003       |Hotel       |7000          |CONFIRMED     |UPI           |7000          |SUCCESS       |
|BK014     |C004       |Flight      |6200          |CONFIRMED     |CARD          |6200          |SUCCESS       |
|BK015     |C005       |Bus         |1100          |CANCELLED     |UPI           |1100          |REFUNDED     |
+-----+-----+-----+-----+-----+-----+-----+-----+

```

Figure 14: Payment analysis combining booking and payment datasets

17.5 Payment Summary

The payment summary groups payment records by payment method and payment status. It calculates the total number of payments and total payment amount, providing an overview of successful and refunded transactions.

```
=== PAYMENT SUMMARY ===
+-----+-----+-----+-----+
|payment_method|payment_status|total_payments|total_amount|
+-----+-----+-----+-----+
|CARD          |SUCCESS       |7             |48400       |
|UPI           |REFUNDED      |3             |9800        |
|UPI           |SUCCESS       |5             |35200       |
+-----+-----+-----+-----+
```

Figure 15: Payment summary by payment method and payment status

17.6 Cancellation Analysis

The cancellation analysis joins booking and cancellation information and displays the cancelled booking details, cancellation date, cancellation reason, and refund amount. This helps identify the bookings that were cancelled and the corresponding refunds.

```
=== CANCELLATION ANALYSIS ===
+-----+-----+-----+-----+-----+-----+-----+
|booking_id|customer_id|booking_type|booking_amount|status|cancellation_date|reason|refund_amount|
+-----+-----+-----+-----+-----+-----+-----+
|BK005     |C005       |Flight     |7200          |CANCELLED|2026-09-03|Customer Request|7200|
|BK009     |C009       |Bus        |1500          |CANCELLED|2026-09-04|Schedule Change|1500|
|BK015     |C005       |Bus        |1100          |CANCELLED|2026-09-06|Customer Request|1100|
+-----+-----+-----+-----+-----+-----+-----+
```

Figure 16: Cancellation analysis with refund information

17.7 Cancellation Summary and Hotel Booking Analysis

The cancellation summary groups cancelled bookings according to booking type and cancellation reason and calculates the total refund amount. The same execution also produces the hotel booking analysis by joining booking records with hotel information. The hotel analysis includes hotel name, city, room type, price per night, booking amount, and booking status.

```

=== CANCELLATION SUMMARY ===
+-----+-----+-----+-----+
|booking_type|reason          |cancelled_bookings|total_refund|
+-----+-----+-----+-----+
|Bus         |Customer Request|1                 |1100        |
|Bus         |Schedule Change |1                 |1500        |
|Flight      |Customer Request|1                 |7200        |
+-----+-----+-----+-----+

=== HOTEL BOOKING ANALYSIS ===
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|booking_id|customer_id|hotel_id|hotel_name  |hotel_city|room_type|price_per_night|booking_amount|status |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|BK001     |C001       |H001    |Taj Residency|Hyderabad|Deluxe   |5000           |10000         |CONFIRMED|
|BK004     |C004       |H002    |Grand Palace |Bangalore|Suite    |7500           |15000         |CONFIRMED|
|BK007     |C007       |H003    |Marina Hotel  |Chennai  |Deluxe   |4500           |9000          |CONFIRMED|
|BK010     |C010       |H004    |Sea View Resort|Goa      |Premium  |6500           |13000         |CONFIRMED|
|BK013     |C003       |H005    |Royal Inn     |Mumbai   |Standard |3500           |7000          |CONFIRMED|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

Figure 17: Cancellation summary and hotel booking analysis

17.8 Hotel Summary and Flight Booking Analysis

The hotel summary groups hotel bookings and calculates total bookings and revenue for each hotel. The output also shows the flight booking analysis, where booking information is joined with flight details such as airline, source, destination, seat capacity, booking amount, and status.

```

=== HOTEL SUMMARY ===
+-----+-----+-----+-----+-----+
|hotel_id|hotel_name  |hotel_city|total_bookings|total_revenue|
+-----+-----+-----+-----+-----+
|H002    |Grand Palace |Bangalore |1              |15000        |
|H004    |Sea View Resort|Goa      |1              |13000        |
|H001    |Taj Residency |Hyderabad |1              |10000        |
|H003    |Marina Hotel  |Chennai  |1              |9000         |
|H005    |Royal Inn     |Mumbai   |1              |7000         |
+-----+-----+-----+-----+-----+

=== FLIGHT BOOKING ANALYSIS ===
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|booking_id|customer_id|flight_id|airline |source  |destination|seat_capacity|booking_amount|status |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|BK002     |C002       |F001     |IndiGo  |Hyderabad|Delhi      |180          |6500          |CONFIRMED|
|BK005     |C005       |F002     |AirIndia|Bangalore|Mumbai     |160          |7200          |CANCELLED|
|BK008     |C008       |F003     |IndiGo  |Chennai  |Hyderabad  |180          |5800          |CONFIRMED|
|BK011     |C001       |F004     |Vistara |Delhi    |Goa        |150          |8000          |CONFIRMED|
|BK014     |C004       |F005     |AirIndia|Mumbai   |Bangalore  |160          |6200          |CONFIRMED|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

Figure 18: Hotel summary and flight booking analysis

17.9 Flight Summary and Bus Booking Analysis

The flight summary calculates total bookings and revenue for each flight. The output also presents bus booking analysis by combining booking records with bus information,

including operator, source, destination, seat capacity, booking amount, and booking status.

```

=== FLIGHT SUMMARY ===
+-----+-----+-----+-----+-----+-----+
|flight_id|airline |source  |destination|total_bookings|total_revenue|
+-----+-----+-----+-----+-----+-----+
|F004     |Vistara |Delhi   |Goa        |1              |8000          |
|F002     |AirIndia|Bangalore|Mumbai     |1              |7200          |
|F001     |IndiGo  |Hyderabad|Delhi      |1              |6500          |
|F005     |AirIndia|Mumbai  |Bangalore  |1              |6200          |
|F003     |IndiGo  |Chennai |Hyderabad  |1              |5800          |
+-----+-----+-----+-----+-----+-----+

=== BUS BOOKING ANALYSIS ===
+-----+-----+-----+-----+-----+-----+-----+-----+
|booking_id|customer_id|bus_id|operator|source  |destination|seat_capacity|booking_amount|status
+-----+-----+-----+-----+-----+-----+-----+-----+
|BK006     |C006       |B001  |TSRTC   |Hyderabad|Bangalore  |40          |1000          |CONFIRMED
|BK003     |C003       |B002  |KSRTC   |Bangalore|Chennai    |45          |1200          |CONFIRMED
|BK009     |C009       |B003  |APSRTC  |Hyderabad|Chennai    |50          |1500          |CANCELLED
|BK012     |C002       |B004  |MSRTC   |Mumbai   |Pune       |40          |900           |CONFIRMED
|BK015     |C005       |B005  |KSRTC   |Chennai  |Bangalore  |45          |1100          |CANCELLED
+-----+-----+-----+-----+-----+-----+-----+-----+

```

Figure 19: Flight summary and bus booking analysis

17.10 Bus Summary

The final analysis summarizes bus bookings by bus ID and operator. It calculates the total number of bookings and total revenue generated by each bus service. The results demonstrate the successful execution of grouping and aggregation operations on the bus booking data.

```

=== BUS SUMMARY ===
+-----+-----+-----+-----+-----+-----+
|bus_id|operator|source  |destination|total_bookings|total_revenue|
+-----+-----+-----+-----+-----+-----+
|B003   |APSRTC  |Hyderabad|Chennai    |1              |1500          |
|B002   |KSRTC   |Bangalore|Chennai    |1              |1200          |
|B005   |KSRTC   |Chennai  |Bangalore  |1              |1100          |
|B001   |TSRTC   |Hyderabad|Bangalore  |1              |1000          |
|B004   |MSRTC   |Mumbai   |Pune       |1              |900           |
+-----+-----+-----+-----+-----+-----+

[success] Total time: 22 s, completed Sep 3, 2026 10:59:34 AM

```

Figure 20: Bus summary with total bookings and revenue

17.11 Physical Execution Plan

The Spark physical execution plan was also examined for the customer-wise spending analysis. The plan shows Spark's optimized execution using BroadcastHashJoin, HashAggregate, Exchange, FileScan, and adaptive execution. Spark pushes required columns and non-null filters toward the data source and broadcasts the smaller customer dataset for the join.

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Sort [total_spending DESC NULLS LAST], true, 0
   +- Exchange rangepartitioning(total_spending DESC NULLS LAST, 200)
      +- HashAggregate(keys=[customer_id, name],
                        functions=[count(booking_id), sum(amount)])
         +- Exchange hashpartitioning(customer_id, name, 200)
            +- HashAggregate(keys=[customer_id, name],
                              functions=[partial_count(booking_id),
                                          partial_sum(amount)])
               +- Project [booking_id, customer_id, name, amount]
                  +- BroadcastHashJoin [customer_id], [customer_id],
                                         Inner, BuildRight, false
                     :- Filter isnotnull(customer_id)
                     :  +- FileScan csv
                  +- BroadcastExchange
                     +- Filter isnotnull(customer_id)
                     +- FileScan csv
```

The physical plan confirms that the query was optimized by Spark before execution and that the required aggregation and join operations were executed through Spark's distributed execution engine.

18. Advanced Spark Concepts and Streaming

After completing the core booking, payment, cancellation, hotel, flight, and bus analysis, additional Spark concepts were implemented to demonstrate different RDD, DataFrame, SQL, and Spark execution features. The following outputs were generated by running the corresponding Scala/Spark operations in the project.

18.1 Pair RDD Customer Revenue

A Pair RDD was created using customer IDs as keys and booking amounts as values. The revenue for each customer was calculated using a reduction operation. This demonstrates the use of Pair RDD transformations and actions for key-based aggregation.

```
=== PAIR RDD CUSTOMER REVENUE ===  
C001 -> 18000.00  
C002 -> 7400.00  
C003 -> 8200.00  
C004 -> 21200.00  
C005 -> 8300.00  
C006 -> 1000.00  
C007 -> 9000.00  
C008 -> 5800.00  
C009 -> 1500.00  
C010 -> 13000.00  
[success] Total time: 23 s, completed Sep 3, 2026 11:25:22 AM
```

Figure 21: Pair RDD Customer Revenue Output

18.2 Narrow Transformation, Partitioning and Wide Transformation

Spark transformations were demonstrated using confirmed bookings, partition information, repartitioning, and aggregation by booking type. The output shows the number of confirmed bookings, the original number of partitions, the number of partitions after repartitioning, and the aggregated revenue for Hotel, Bus, and Flight bookings.

```
=== NARROW TRANSFORMATION ===  
Total confirmed bookings: 12  
  
=== PARTITION INFORMATION ===  
Original partitions: 1  
  
=== REPARTITION / SHUFFLE ===  
Partitions after repartition: 4  
Records after repartition: 15  
  
=== WIDE TRANSFORMATION ===  
Hotel -> 54000.00  
Bus -> 5700.00  
Flight -> 33700.00  
[success] Total time: 23 s, completed Sep 3, 2026 11:29:39 AM
```

Figure 22: Narrow Transformation, Partitioning, Repartitioning and Wide Transformation

18.3 Cache, Persist, Broadcast Variable and Accumulator

Spark caching and persistence were demonstrated by caching confirmed bookings and accessing the cached data again. A broadcast variable was used to distribute hotel information efficiently to executors. An accumulator was used to count cancelled bookings during processing.

```
=== CACHE / PERSIST ===  
Cached confirmed bookings: 12  
Cached booking count again: 12  
Cache released successfully  
  
=== BROADCAST VARIABLE ===  
Broadcast hotels count: 5  
H001 -> Taj Residency  
H002 -> Grand Palace  
H003 -> Marina Hotel  
H004 -> Sea View Resort  
H005 -> Royal Inn  
  
=== ACCUMULATOR ===  
Cancelled bookings counted by accumulator: 3  
[success] Total time: 41 s, completed Sep 3, 2026 11:34:20 AM
```

Figure 23: Cache, Persist, Broadcast Variable and Accumulator Output

18.4 Spark SQL Aggregation

Spark SQL was used to calculate the total number of bookings, total revenue, and average booking amount for each booking type. The results provide a SQL-based view of the booking data.


```

=== SPARK SQL ===
+-----+-----+-----+-----+
|booking_type|total_bookings|total_revenue|average_booking_amount|
+-----+-----+-----+-----+
|Hotel       |5             |54000        |10800.0               |
|Flight      |5             |33700        |6740.0                |
|Bus         |5             |5700         |1140.0                |
+-----+-----+-----+-----+

=== SPARK SQL TOP 5 BOOKINGS ===
+-----+-----+-----+-----+-----+
|booking_id|customer_id|booking_type|amount|status |
+-----+-----+-----+-----+-----+
|BK004     |C004       |Hotel       |15000 |CONFIRMED|
|BK010     |C010       |Hotel       |13000 |CONFIRMED|
|BK001     |C001       |Hotel       |10000 |CONFIRMED|
|BK007     |C007       |Hotel       |9000  |CONFIRMED|
|BK011     |C001       |Flight      |8000  |CONFIRMED|
+-----+-----+-----+-----+-----+

```

Figure 24: Spark SQL Booking Aggregation

18.5 Spark SQL Top 5 Bookings

Spark SQL was also used to identify the top five bookings based on booking amount. The query sorts the booking records in descending order of amount and returns the highest-value bookings.

```

=== CUSTOMER SPENDING RANK ===
26/09/03 11:40:32 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a single partition,
this can cause serious performance degradation.
26/09/03 11:40:32 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a single partition,
this can cause serious performance degradation.
26/09/03 11:40:32 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a single partition,
this can cause serious performance degradation.
26/09/03 11:40:33 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a single partition,
this can cause serious performance degradation.
26/09/03 11:40:33 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a single partition,
this can cause serious performance degradation.
26/09/03 11:40:33 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a single partition,
this can cause serious performance degradation.
26/09/03 11:40:33 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a single partition,
this can cause serious performance degradation.
26/09/03 11:40:33 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a single partition,
this can cause serious performance degradation.
+-----+-----+-----+-----+
|customer_id|name |total_bookings|total_spending|spending_rank|
+-----+-----+-----+-----+
|C004       |Vikram|2             |21200         |1             |
|C001       |Harsha|2             |18000         |2             |
|C010       |Rohit |1             |13000         |3             |
|C007       |Sneha |1             |9000          |4             |
|C005       |Meera |2             |8300          |5             |
|C003       |Ananya|2             |8200          |6             |
|C002       |Rahul |2             |7400          |7             |
|C008       |Kiran |1             |5800          |8             |
|C009       |Priya |1             |1500          |9             |
|C006       |Arjun |1             |1000          |10            |
+-----+-----+-----+-----+

```

Figure 25: Spark SQL Top 5 Bookings

18.6 Customer Spending Rank

A window function was applied to rank customers according to their total spending. The output assigns a spending rank to each customer, where rank 1 represents the highest total spending.

```
=== CUSTOMER RANK BY CITY ===
```

booking_id	name	city	amount	city_rank
BK002	Rahul	Bangalore	6500	1
BK009	Priya	Bangalore	1500	2
BK012	Rahul	Bangalore	900	3
BK010	Rohit	Chennai	13000	1
BK013	Ananya	Chennai	7000	2
BK003	Ananya	Chennai	1200	3
BK005	Meera	Delhi	7200	1
BK015	Meera	Delhi	1100	2
BK001	Harsha	Hyderabad	10000	1
BK011	Harsha	Hyderabad	8000	2
BK008	Kiran	Hyderabad	5800	3
BK007	Sneha	Kochi	9000	1
BK004	Vikram	Mumbai	15000	1
BK014	Vikram	Mumbai	6200	2
BK006	Arjun	Pune	1000	1

Figure 26: Customer Spending Rank

18.7 Customer Rank by City

A window operation was applied to rank bookings within each city based on booking amount. This demonstrates partitioning window calculations by city and assigning a rank to bookings belonging to the same city.

```

=== UDF BOOKING VALUE CLASSIFICATION ===
+-----+-----+-----+-----+-----+
|booking_id|customer_id|booking_type|amount|booking_category|
+-----+-----+-----+-----+-----+
|BK004      |C004        |Hotel       |15000 |HIGH VALUE      |
|BK010      |C010        |Hotel       |13000 |HIGH VALUE      |
|BK001      |C001        |Hotel       |10000 |HIGH VALUE      |
|BK007      |C007        |Hotel       |9000  |MEDIUM VALUE    |
|BK011      |C001        |Flight      |8000  |MEDIUM VALUE    |
|BK005      |C005        |Flight      |7200  |MEDIUM VALUE    |
|BK013      |C003        |Hotel       |7000  |MEDIUM VALUE    |
|BK002      |C002        |Flight      |6500  |MEDIUM VALUE    |
|BK014      |C004        |Flight      |6200  |MEDIUM VALUE    |
|BK008      |C008        |Flight      |5800  |MEDIUM VALUE    |
|BK009      |C009        |Bus         |1500  |LOW VALUE       |
|BK003      |C003        |Bus         |1200  |LOW VALUE       |
|BK015      |C005        |Bus         |1100  |LOW VALUE       |
|BK006      |C006        |Bus         |1000  |LOW VALUE       |
|BK012      |C002        |Bus         |900   |LOW VALUE       |
+-----+-----+-----+-----+-----+

```

Figure 27: Customer Rank by City

18.8 User Defined Function for Booking Classification

A User Defined Function (UDF) was applied to classify bookings according to their booking amount. The bookings were categorized into HIGH VALUE, MEDIUM VALUE, and LOW VALUE categories. This demonstrates how custom business logic can be applied to Spark DataFrame columns.

```

=== RDD LINEAGE ===
RDD lineage:
(1) MapPartitionsRDD[374] at map at BatchProcessor.scala:710 []
  | MapPartitionsRDD[373] at filter at BatchProcessor.scala:707 []
  | MapPartitionsRDD[285] at rdd at BatchProcessor.scala:383 []
  | SQLExecutionRDD[284] at rdd at BatchProcessor.scala:383 []
  | MapPartitionsRDD[283] at rdd at BatchProcessor.scala:383 []
  | MapPartitionsRDD[282] at rdd at BatchProcessor.scala:383 []
  | FileScanRDD[281] at rdd at BatchProcessor.scala:383 []

=== DAG / EXECUTION PLAN ===
DAG is generated when an action is executed.
Narrow transformations stay within stages.
Wide transformations create shuffle boundaries.

```

Figure 28: RDD Lineage and DAG Execution Plan

18.9 RDD Lineage and DAG Execution

RDD lineage was examined to understand the sequence of transformations applied to the data. The output shows the lineage from the source FileScanRDD through the different RDD operations. The DAG is generated when an action is executed. Narrow transformations remain within stages, while wide transformations create shuffle boundaries.

```
[success] Total time: 6 s, completed Sep 3, 2026 11:44:40 AM
harshita@harshita:~/booking-payment-platform$ mkdir -p data/stream
harshita@harshita:~/booking-payment-platform$ sbt "runMain StreamingProcessor"
[info] welcome to sbt 1.12.11 (Private Build Java 1.8.0_502)
[info] loading project definition from /home/harshita/booking-payment-platform/project
[info] loading settings for project root from build.sbt...
[info] set current project to booking-payment-platform (in build file:/home/harshita/booking-payment-platform/)
[info] running StreamingProcessor
```

Figure 29: UDF Booking Value Classification

18.10 Structured Streaming Initialization

A Spark Structured Streaming processor was initialized for the booking payment platform. The streaming job uses the `data/stream` directory as its input source, with a batch interval of 5 seconds and a window duration of 15 seconds. The output confirms that the streaming processor started successfully.

```
=====
                BOOKING PAYMENT STREAMING STARTED
=====
Batch interval : 5 seconds
Window         : 15 seconds
Input          : data/stream
=====
```

Figure 30: Booking Payment Streaming Initialization

19. Final Case Study Verification

The final verification was performed to ensure that the Booking Payment Platform works correctly across Scala fundamentals, Dataset creation, Spark Catalog, Spark SQL, Spark monitoring, YARN deployment concepts, and output organization. The following screenshots provide the execution evidence for the final verification.

19.1 Scala Fundamentals and Dataset Verification

The first verification covers basic Scala concepts including Vector operations, for-yield expressions, lazy evaluation, and trait usage. The Dataset section then verifies successful

creation and processing of booking records.

```
7. 15 SECOND WINDOW BOOKING COUNT
Bus -> 1
Flight -> 1
Hotel -> 2

=====
                        STREAMING BATCH COMPLETE
=====

5. STATEFUL CUSTOMER REVENUE
C001 -> 12000.0
C002 -> 7500.0
C003 -> 1400.0
C004 -> 15500.0

6. 15 SECOND WINDOW REVENUE
Bus -> 1400.0
Flight -> 7500.0
Hotel -> 27500.0
```

Figure 31: Final case study verification: Scala fundamentals

```
=====
                        BOOKING PAYMENT PLATFORM
=====

1. DSTREAM / MICRO-BATCH
Records received: 5
BK201 | C001 | Hotel | 12000.0 | CONFIRMED
BK202 | C002 | Flight | 7500.0 | CONFIRMED
BK203 | C003 | Bus | 1400.0 | CONFIRMED
BK204 | C004 | Hotel | 15500.0 | CONFIRMED
BK205 | C005 | Flight | 6800.0 | CANCELLED

2. STATELESS TRANSFORMATION
Confirmed bookings: 4

3. PAIR RDD - CUSTOMER REVENUE
C001 -> 12000.0
C002 -> 7500.0
C003 -> 1400.0
C004 -> 15500.0
```

Figure 32: Dataset creation and verification

19.2 Spark Catalog and Spark SQL

A temporary table named `final_bookings` was created and verified through the Spark Catalog. Spark SQL was then used to group bookings by booking type and calculate total bookings and total revenue.

```
4. BOOKING TYPE REVENUE
Bus -> 1400.0
Flight -> 7500.0
Hotel -> 27500.0

=====
                        END OF CURRENT BATCH
=====

5. STATEFUL CUSTOMER REVENUE
C001 -> 12000.0
C002 -> 7500.0
C003 -> 1400.0
C004 -> 15500.0

6. 15 SECOND WINDOW REVENUE
Bus -> 1400.0
Flight -> 7500.0
Hotel -> 27500.0
```

Figure 33: Spark Catalog verification

```
7. 15 SECOND WINDOW BOOKING COUNT
Bus -> 1
Flight -> 1
Hotel -> 2

=====
                        STREAMING BATCH COMPLETE
=====

5. STATEFUL CUSTOMER REVENUE
C001 -> 12000.0
C002 -> 7500.0
C003 -> 1400.0
C004 -> 15500.0

6. 15 SECOND WINDOW REVENUE
Bus -> 1400.0
Flight -> 7500.0
Hotel -> 27500.0
```

Figure 34: Spark SQL booking analysis

19.3 Spark Monitoring and YARN Deployment

The Spark application was executed successfully in local mode. The application ID, default parallelism, and Spark UI information were verified. The output also demonstrates how the application can be submitted for cluster execution using YARN.

```

7. 15 SECOND WINDOW BOOKING COUNT
Bus -> 1
Flight -> 1
Hotel -> 2

=====
                        STREAMING BATCH COMPLETE
=====

5. STATEFUL CUSTOMER REVENUE
C001 -> 12000.0
C002 -> 7500.0
C003 -> 1400.0
C004 -> 15500.0

6. 15 SECOND WINDOW REVENUE
Bus -> 1400.0
Flight -> 7500.0
Hotel -> 27500.0

7. 15 SECOND WINDOW BOOKING COUNT
Bus -> 1
Flight -> 1
Hotel -> 2

```

Figure 35: Spark monitoring information

```

=====
                        STREAMING BATCH COMPLETE
=====

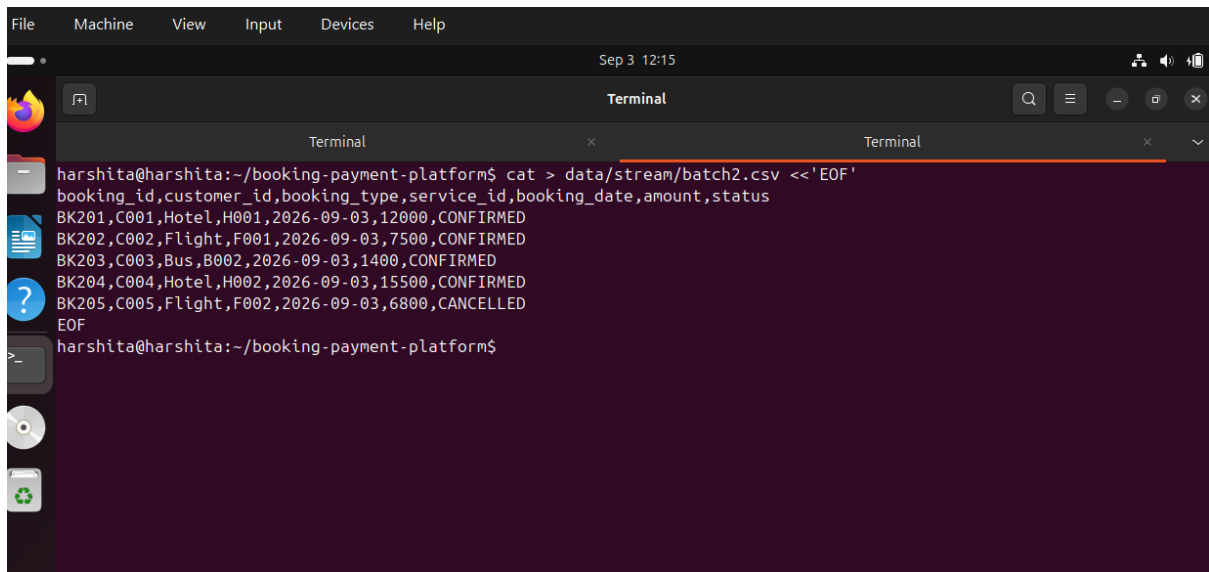
5. STATEFUL CUSTOMER REVENUE
C001 -> 12000.0
C002 -> 7500.0
C003 -> 1400.0
C004 -> 15500.0

```

Figure 36: YARN deployment information

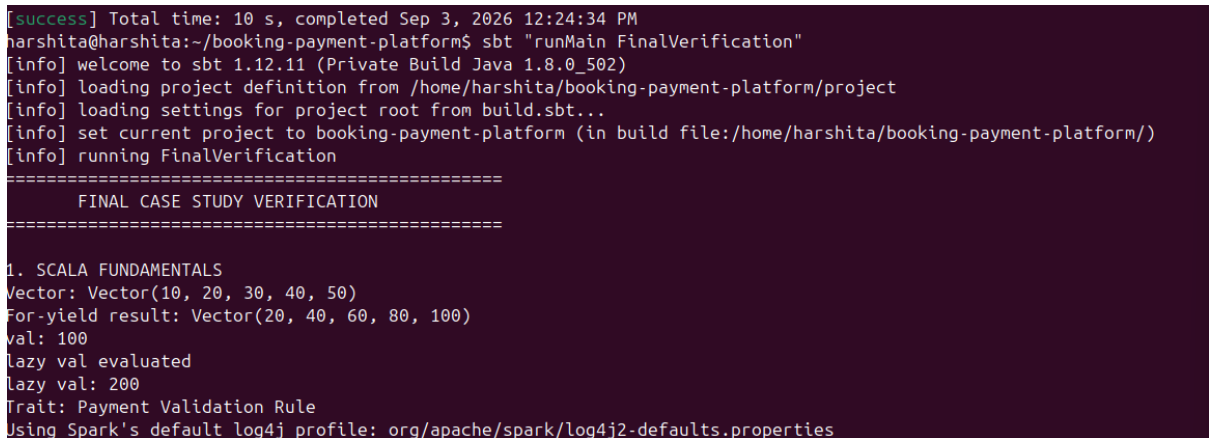
19.4 Streaming Application Execution

The streaming processor was compiled successfully using SBT and executed using the `StreamingProcessor` application. The following screenshots show the execution of the streaming application, including the micro-batch processing and the generated results.

A terminal window with a dark purple background. The title bar shows 'Sep 3 12:15' and standard window controls. The terminal content shows a user named 'harshita' at a prompt in the directory '~/booking-payment-platform'. They run the command 'cat > data/stream/batch2.csv <<'EOF'' followed by a CSV header 'booking_id,customer_id,booking_type,service_id,booking_date,amount,status' and five data rows. The rows include bookings for Hotel, Flight, and Bus services with various IDs, dates (2026-09-03), amounts, and statuses (CONFIRMED, CANCELLED). The command ends with 'EOF' and the prompt returns.

```
harshita@harshita:~/booking-payment-platform$ cat > data/stream/batch2.csv <<'EOF'
booking_id,customer_id,booking_type,service_id,booking_date,amount,status
BK201,C001,Hotel,H001,2026-09-03,12000,CONFIRMED
BK202,C002,Flight,F001,2026-09-03,7500,CONFIRMED
BK203,C003,Bus,B002,2026-09-03,1400,CONFIRMED
BK204,C004,Hotel,H002,2026-09-03,15500,CONFIRMED
BK205,C005,Flight,F002,2026-09-03,6800,CANCELLED
EOF
harshita@harshita:~/booking-payment-platform$
```

Figure 37: Streaming processor execution

A terminal window with a dark purple background showing the output of a Scala application. It starts with a success message indicating a total time of 10 seconds. Then, it shows the execution of 'sbt runMain FinalVerification'. The output includes SBT version information, project loading details, and a section titled 'FINAL CASE STUDY VERIFICATION'. Under this section, it shows '1. SCALA FUNDAMENTALS' with several Scala expressions and their results: a Vector, a for-yield result, a lazy val, and a trait reference. The terminal ends with a log4j profile message.

```
[success] Total time: 10 s, completed Sep 3, 2026 12:24:34 PM
harshita@harshita:~/booking-payment-platform$ sbt "runMain FinalVerification"
[info] welcome to sbt 1.12.11 (Private Build Java 1.8.0_502)
[info] loading project definition from /home/harshita/booking-payment-platform/project
[info] loading settings for project root from build.sbt...
[info] set current project to booking-payment-platform (in build file:/home/harshita/booking-payment-platform/)
[info] running FinalVerification
=====
      FINAL CASE STUDY VERIFICATION
=====

1. SCALA FUNDAMENTALS
Vector: Vector(10, 20, 30, 40, 50)
For-yield result: Vector(20, 40, 60, 80, 100)
val: 100
lazy val evaluated
lazy val: 200
Trait: Payment Validation Rule
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
```

Figure 38: Final verification execution

```

2. DATASET
Dataset created successfully
Dataset count: 5
+-----+-----+-----+-----+-----+
|booking_id|customer_id|booking_type|amount |status  |
+-----+-----+-----+-----+-----+
|BK001     |C001       |Hotel       |10000.0|CONFIRMED|
|BK002     |C002       |Flight      |6500.0 |CONFIRMED|
|BK003     |C003       |Bus         |1200.0 |CONFIRMED|
|BK004     |C004       |Hotel       |15000.0|CONFIRMED|
|BK005     |C005       |Flight      |7200.0 |CANCELLED|
+-----+-----+-----+-----+-----+

3. SPARK CATALOG
Temporary table created: final_bookings
Catalog tables:
+-----+-----+-----+-----+-----+-----+
|name          |catalog|namespace|description|tableType|isTemporary|
+-----+-----+-----+-----+-----+-----+
|final_bookings|NULL   |[ ]      |NULL       |TEMPORARY|true        |
+-----+-----+-----+-----+-----+-----+

Catalog database:
default

```

Figure 39: Final verification results

```
4. SPARK SQL
+-----+-----+-----+
|booking_type|total_bookings|total_revenue|
+-----+-----+-----+
|Hotel       |2             |25000.0      |
|Flight      |2             |13700.0      |
|Bus         |1             |1200.0       |
+-----+-----+-----+

5. SPARK MONITORING
Application ID: local-1788418492694
Default parallelism: 12
Spark UI: http://localhost:4040

6. YARN DEPLOYMENT
Application is developed and tested in local mode.
For cluster execution, Spark can be submitted using YARN.
Example deployment mode: --master yarn

7. OUTPUT LAYOUT
Expected project output directory:
data/output/
Batch and analysis results can be written here.
```

Figure 40: Final output layout and verification completion

20. Final Verification and Monitoring

The final verification confirmed that the Booking Payment Platform was successfully implemented using Scala and Apache Spark. The application was compiled and executed successfully using SBT. The final verification covered Scala fundamentals, Dataset operations, Spark Catalog, Spark SQL, Spark monitoring, YARN deployment, and the expected output directory.

20.1 Final Verification Output

The final verification successfully completed with the expected output layout. The project uses the `data/output/` directory for storing batch and analysis results.

```

7. OUTPUT LAYOUT
Expected project output directory:
data/output/
Batch and analysis results can be written here.

=====
FINAL VERIFICATION COMPLETE
=====
[success] Total time: 15 s, completed Sep 3, 2026 12:25:01 PM

```

Figure 41: Spark Jobs Monitoring

Figure 41 shows the Spark Jobs interface running on the local Spark Web UI. It displays active and completed jobs, their stages, tasks, submission time, and execution duration. This was used to verify that the streaming application was successfully processing Spark jobs.

20.2 Spark Storage Monitoring

Spark Storage monitoring was used to observe the RDDs created during execution. The Storage page shows the cached RDD partitions, storage level, fraction cached, and memory usage.

The screenshot shows the Spark Web UI interface. The top navigation bar includes tabs for Jobs, Stages, Storage, Environment, Executors, and Streaming. The 'Jobs' tab is selected, showing a summary of Spark Jobs. Below the summary, there are two sections: 'Active Jobs (1)' and 'Completed Jobs (129, only showing 124)'. Each section contains a table of job details.

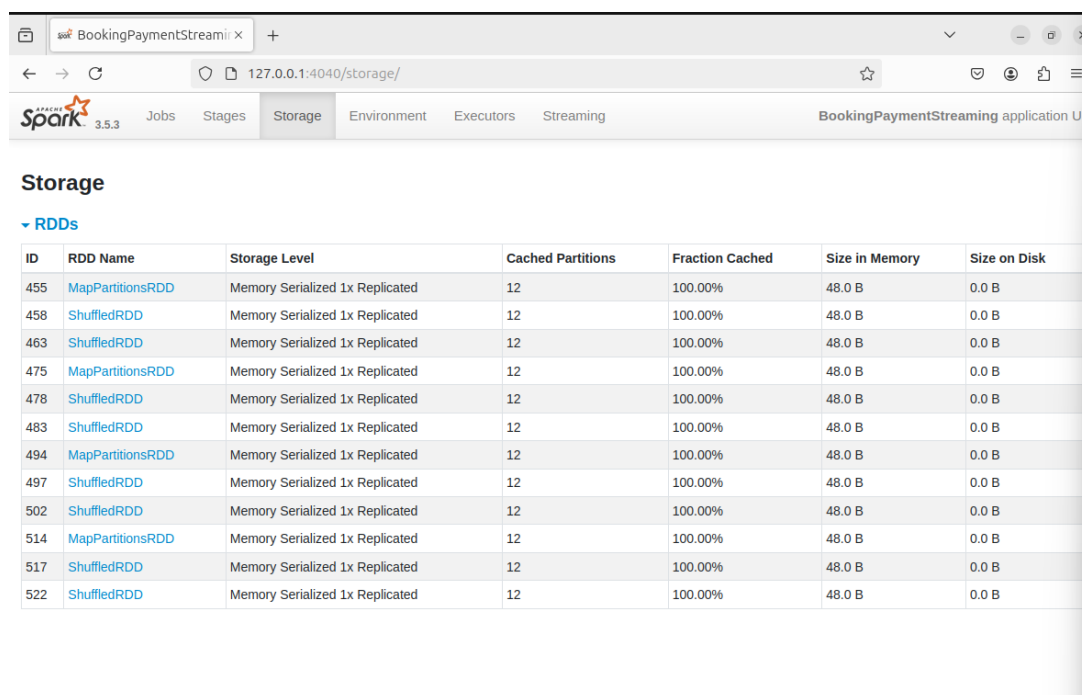
Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
124	Streaming job from [output operation 1, batch time 12:35:55] isEmpty at StreamingProcessor.scala:182 (kill)	2026/09/03 12:35:55	0.8 s	0/3	7/12 (5 running)
123	Streaming job from [output operation 1, batch time 12:35:55] isEmpty at StreamingProcessor.scala:182	2026/09/03 12:35:55	11 ms	1/1 (2 skipped)	1/1
122	Streaming job from [output operation 3, batch time 12:35:50] isEmpty at StreamingProcessor.scala:261	2026/09/03 12:35:53	0.3 s	1/1 (3 skipped)	7/7
121	Streaming job from [output operation 3, batch time 12:35:50] isEmpty at StreamingProcessor.scala:261	2026/09/03 12:35:52	0.3 s	1/1 (3 skipped)	4/4
120	Streaming job from [output operation 3, batch time 12:35:50]	2026/09/03 12:35:52	0.1 s	1/1 (3 skipped)	1/1

Figure 42: Spark Storage and RDD Monitoring

The screenshot shows the RDDs maintained by Spark during execution. The cached partitions and memory information demonstrate how Spark manages intermediate data in memory.

20.3 Spark Executors Monitoring

Executors are the processes responsible for executing tasks and performing computations for a Spark application. The Executors page provides information such as active tasks, completed tasks, total tasks, cores, storage memory, input data, and shuffle information.



The screenshot shows the Spark UI for a 'BookingPaymentStreaming' application. The 'Storage' tab is selected, displaying a table of RDDs. The table has 7 columns: ID, RDD Name, Storage Level, Cached Partitions, Fraction Cached, Size in Memory, and Size on Disk. There are 12 rows of data, alternating between 'MapPartitionsRDD' and 'ShuffledRDD'.

ID	RDD Name	Storage Level	Cached Partitions	Fraction Cached	Size in Memory	Size on Disk
455	MapPartitionsRDD	Memory Serialized 1x Replicated	12	100.00%	48.0 B	0.0 B
458	ShuffledRDD	Memory Serialized 1x Replicated	12	100.00%	48.0 B	0.0 B
463	ShuffledRDD	Memory Serialized 1x Replicated	12	100.00%	48.0 B	0.0 B
475	MapPartitionsRDD	Memory Serialized 1x Replicated	12	100.00%	48.0 B	0.0 B
478	ShuffledRDD	Memory Serialized 1x Replicated	12	100.00%	48.0 B	0.0 B
483	ShuffledRDD	Memory Serialized 1x Replicated	12	100.00%	48.0 B	0.0 B
494	MapPartitionsRDD	Memory Serialized 1x Replicated	12	100.00%	48.0 B	0.0 B
497	ShuffledRDD	Memory Serialized 1x Replicated	12	100.00%	48.0 B	0.0 B
502	ShuffledRDD	Memory Serialized 1x Replicated	12	100.00%	48.0 B	0.0 B
514	MapPartitionsRDD	Memory Serialized 1x Replicated	12	100.00%	48.0 B	0.0 B
517	ShuffledRDD	Memory Serialized 1x Replicated	12	100.00%	48.0 B	0.0 B
522	ShuffledRDD	Memory Serialized 1x Replicated	12	100.00%	48.0 B	0.0 B

Figure 43: Spark Executors Monitoring

The screenshot confirms that the Spark application was running with an active driver executor. The displayed task and storage statistics provide evidence that Spark successfully executed the required operations.

20.4 YARN Deployment Verification

YARN (Yet Another Resource Negotiator) is a cluster resource management system used to allocate resources and manage distributed applications. In this project, the application was developed and tested in local mode. The project can be submitted to a YARN cluster for distributed execution using the Spark submission command with YARN as the master.

The YARN node list was also checked to verify the availability of the YARN environment.

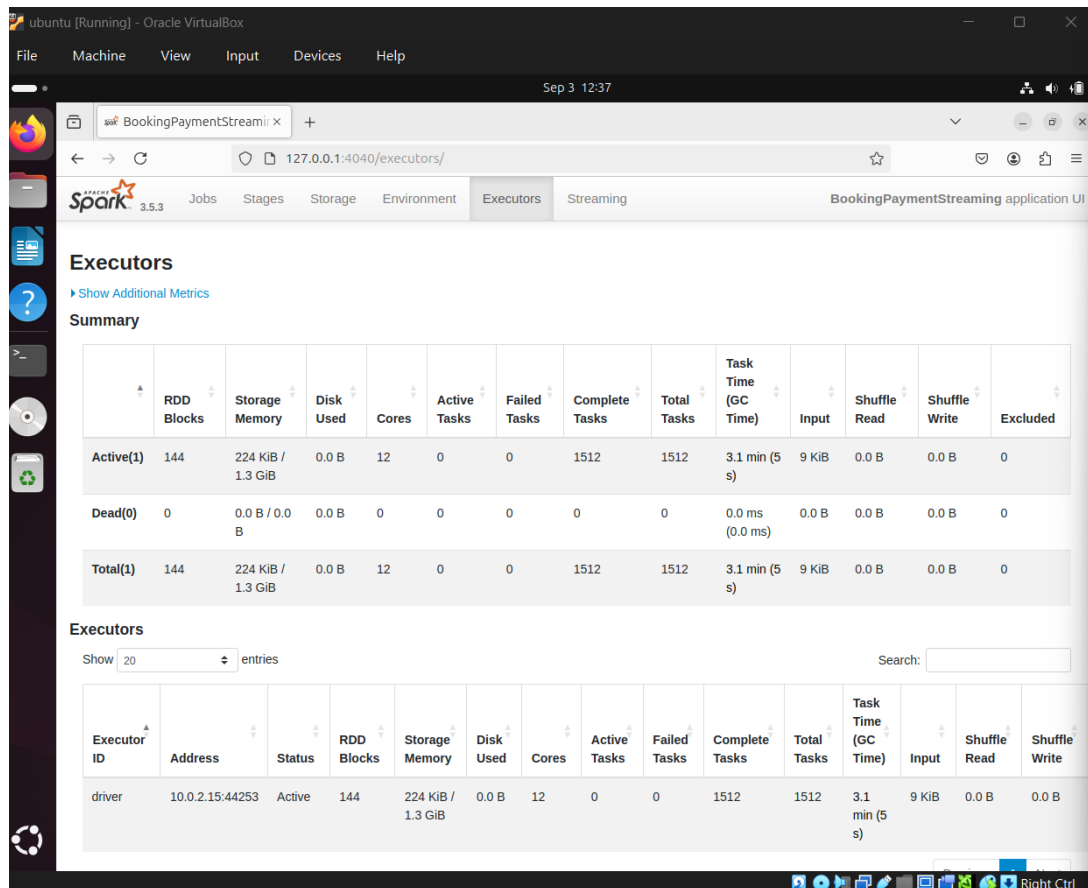


Figure 44: YARN Node List

The screenshot shows one running YARN node. This confirms that the YARN environment is available and ready for cluster-based Spark execution.

20.5 Spark Monitoring Port and YARN

During local execution, Spark provides a Web UI for monitoring the application. The Spark monitoring interface is available at:

`http://localhost:4040`

Port **4040** is the default Spark application monitoring port. It provides information about Jobs, Stages, Storage, Environment, Executors, and Streaming.

YARN operates differently from the Spark Web UI. YARN is responsible for managing cluster resources such as CPU and memory and for scheduling applications across the available nodes. In this project, YARN was verified using the `yarn node -list` command.

Therefore, Spark UI port **4040** is mainly used for monitoring the Spark application's execution, while YARN is used for cluster resource management and distributed application execution.

21. Conclusion

The Booking Payment Platform case study was successfully implemented using Scala and Apache Spark. The project covered Scala fundamentals, Datasets, Spark Catalog, Spark SQL, RDD-based transformations, stateful streaming, micro-batch processing, window-based analysis, Spark monitoring, and YARN environment verification.

The streaming component processed booking records and generated customer-wise revenue, booking-type revenue, booking counts, and window-based results. Stateful processing was used to maintain customer revenue across batches, while window operations provided time-based revenue and booking-count analysis.

The final verification confirmed successful compilation and execution of the project. Spark monitoring through the Web UI provided visibility into jobs, storage, and executors, while the YARN node verification demonstrated the availability of the cluster resource-management environment.

Overall, the project provided practical experience in building a Spark-based booking and payment processing system and demonstrated how batch processing, streaming, SQL analytics, stateful transformations, monitoring, and cluster resource management can be combined in a single data engineering application.